AI ASSISTED CODING

LAB ASSIGNMENT-9.1
Name: A.Deepthi
Ht.No: 2303A52390
Batch: 32

Consider the following Python function:
def find_max(numbers):
return max(numbers)
Task:
• Write documentation for the function in all three formats:
(a) Docstring
(b) Inline comments
(c) Google-style documentation
• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.
• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer

```
Untitled-1.py ×   attendance.py   attendance.html   24 feb.py
Untitled-1.py > ...
def find_max(numbers):
    """
    This function takes a list of numbers and returns the maximum value.
    Parameters:
    numbers (list): A list of numerical values.
    Returns:
    The maximum value in the list of numbers.
    """

    """
    Args:    numbers (list): A list of numerical values.
    Returns:    The maximum value in the list of numbers.
    Raises:    ValueError: If the input is not a list of numbers.
    """
    return max(numbers) # This function takes a list of numbers and returns the maximum value using the built-in max() function.
input_list = [1, 2, 3, 4, 5]
print(find_max(input_list))
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASSISTENCE/Untitled-1.py"
5
```

Inline Comments

Advantages:

- Simple and quick.

- Useful for explaining specific lines.

Disadvantages:

- No structure.
- Cannot clearly document parameters or exceptions.
- Not suitable for reusable libraries.

Best Use Case:
Small scripts or explaining complex logic inside a function.

Basic Docstring

Advantages:

- Clear overview of function purpose.
- Easy to write and understand.

Disadvantages:

- No strict structure.
- Exceptions are often unclear.

Best Use Case:
Small projects or beginner-level code.

Google-Style Docstring

Advantages:

- Structured and consistent.
- Clearly documents arguments, return types, and exceptions.
- Suitable for auto-generated documentation tools.

Disadvantages:

- Slightly more verbose.

Best Use Case:
Professional projects, libraries, APIs, and large systems.

Use **Google-style documentation**.

Reason: A mathematical utilities library will be reused by many developers. Functions must clearly define input types, return types, and possible errors. Structured documentation prevents misuse and improves maintainability.

Problem 2: Consider the following Python function:

```
def login(user, password, credentials):

return credentials.get(user) == password
```

Task:

1. Write documentation in all three formats.

2. Critically compare the approaches.

3. Recommend which style would be most helpful for new

developers onboarding a project, and justify your choice.

```python
def login(user,password,credentials):
    """
    This function takes a username, password, and a dictionary of credentials and checks if the username and password match any entry in the credentials dictionary.
    Parameters:
    user (str): The username to be checked.
    password (str): The password to be checked.
    credentials (dict): A dictionary containing username-password pairs.
    Returns:
    A string indicating whether the login was successful or not.
    """
    """
    Args:    user (str): The username to be checked.
    password (str): The password to be checked.
    credentials (dict): A dictionary containing username-password pairs.
    Returns:    A string indicating whether the login was successful or not.
    Raises:    KeyError: If the username is not found in the credentials dictionary.
    """
    if not isinstance(credentials, dict):
        raise ValueError("Credentials must be a dictionary")
    if user in credentials and credentials[user] == password:# This function checks if the provided username exists in the credentials
        #dictionary and if the corresponding password matches the provided password.
        return "Login successful"
    else:
        return "Login failed"
input_credentials = {"user1": "password1", "user2": "password2"}
print(login("user1", "password1", input_credentials))
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admi
rams\Python\Python312\python.exe "c:/Users/Admin/Downloads
led-1.py"
Login successful
```

Inline Comments

Advantages:

- Very simple.
- Useful for explaining a single line of logic.

Disadvantages:

- No structure.
- Does not clearly define inputs or return type.
- Not helpful for understanding function contract.

Use Case:
Small scripts or quick internal code.

Basic Docstring

Advantages:

- Gives a clear description of purpose.
- Easy to write and read.

Disadvantages:

- No strict structure.
- Exceptions and return types may not be clearly defined.

Use Case:
Small or medium projects.

Google Style

Advantages:

- Clearly separates arguments, returns, and errors.
- Easy to scan and consistent.
- Good for team projects.

Disadvantages:

- Slightly longer to write.

Use Case:
Professional systems, APIs, shared libraries.

Use **Google-style documentation**.

Reason: New developers need clarity about inputs, outputs, and possible errors. Structured documentation reduces confusion and speeds up onboarding.

Problem 3: Calculator (Automatic Documentation Generation)
Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.
Instructions:
1. Create a Python module calculator.py that includes the following functions, each written with appropriate docstrings:
o add(a, b) – returns the sum of two numbers
o subtract(a, b) – returns the difference of two numbers
o multiply(a, b) – returns the product of two numbers
o divide(a, b) – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

```python
def add(a,b):
    """
    this function takes two numbers as input and returns their sum
    Parameters:
    a (int or float): the first number
    b (int or float): the second number
    """
    return a+b
def subtract(a,b):
    """
    this function takes two numbers as input and returns their difference
    Parameters:
    a (int or float): the first number
    b (int or float): the second number
    """
    return a-b
def division(a,b):
    """
    this function takes two numbers as input and returns their quotient
    Parameters:
    a (int or float): the first number
    b (int or float): the second number
    """
    if b==0:
        return "division by zero is not allowed"
    else:
        return a/b
print(add.__doc__)
print(subtract.__doc__)
print(division.__doc__)
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASS
ISTENCE/calculatorexample.py"

    this function takes two numbers as input and returns their sum
    Parameters:
    a (int or float): the first number
    b (int or float): the second number


    this function takes two numbers as input and returns their difference
    Parameters:
    a (int or float): the first number
    b (int or float): the second number


    this function takes two numbers as input and returns their quotient
    Parameters:
    a (int or float): the first number
    b (int or float): the second number

PS C:\Users\Admin\Downloads\AI ASSISTENCE>
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc calculator

    this function takes two numbers as input and returns their sum
    Parameters:
    a (int or float): the first number
    b (int or float): the second number


    this function takes two numbers as input and returns their difference
    Parameters:
    a (int or float): the first number
    b (int or float): the second number


    this function takes two numbers as input and returns their quotient
    Parameters:
    a (int or float): the first number
    b (int or float): the second number

Help on module calculator:

NAME
    calculator

DESCRIPTION
    this module include four function and
    their implementation
    and also includes a docstring to explain the usage of each function

FUNCTIONS
    add(a, b)
        this function takes two numbers as input and returns their sum
        Parameters:
        a (int or float): the first number
        b (int or float): the second number

    division(a, b)
        this function takes two numbers as input and returns their quotient
```

```
        a (int or float): the first number
        b (int or float): the second number

    subtract(a, b)
        this function takes two numbers as input and returns their difference
        Parameters:
        a (int or float): the first number
        b (int or float): the second number

FILE
    c:\users\admin\downloads\ai assistence\calculator.py


PS C:\Users\Admin\Downloads\AI ASSISTENCE>
PS C:\Users\Admin\Downloads\AI ASSISTENCE>
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -w calculator

    this function takes two numbers as input and returns their sum
    Parameters:
    a (int or float): the first number
    b (int or float): the second number


    this function takes two numbers as input and returns their difference
    Parameters:
    a (int or float): the first number
    b (int or float): the second number


    this function takes two numbers as input and returns their quotient
    Parameters:
    a (int or float): the first number
    b (int or float): the second number

wrote calculator.html
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -p 2429
Server ready at http://localhost:2429/
Server commands: [b]rowser, [q]uit
server> b
server>
```

Problem 4: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

o decimal_to_hexadecimal(n)

2. Use Copilot for auto-generating docstrings.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

```python
def decimal_to_binary(n):
    """
    This function takes a decimal number as input and returns its binary representation as a string
    """
    if n == 0:
        return "0"
    binary = ""
    while n > 0:
        binary = str(n % 2) + binary
        n = n // 2
    return binary
print(decimal_to_binary(10)) # Output: 1010
print(decimal_to_binary(0)) # Output: 0
def binary_to_decimal(b):
    """
    This function takes a binary number as input and returns its decimal representation as an integer
    """
    decimal = 0
    for i in range(len(b)):
        decimal += int(b[-(i+1)]) * (2 ** i)
    return decimal
print(binary_to_decimal("1010")) # Output: 10
print(binary_to_decimal("0")) # Output: 0
def decimal_to_hexadecimal(n):
    """
    This function takes a decimal number as input and returns its hexadecimal representation as a string
    """
    if n == 0:
        return "0"
    hexadecimal = ""
    hex_digits = "0123456789ABCDEF"
    while n > 0:
        hexadecimal = hex_digits[n % 16] + hexadecimal
        n = n // 16
    return hexadecimal
print(decimal_to_hexadecimal(255)) # Output: FF
print(decimal_to_hexadecimal(0)) # Output: 0
```

Ln 40, Col 38 Spaces: 4 UTF-8 CRLF { } Python

```python
def decimal_to_hexadecimal(n):
    """
    This function takes a decimal number as input and returns its hexadecimal representation as a string
    """
    if n == 0:
        return "0"
    hexadecimal = ""
    hex_digits = "0123456789ABCDEF"
    while n > 0:
        hexadecimal = hex_digits[n % 16] + hexadecimal
        n = n // 16
    return hexadecimal
print(decimal_to_hexadecimal(255)) # Output: FF
print(decimal_to_hexadecimal(0)) # Output: 0
print(binary_to_decimal.__doc__)
print(decimal_to_binary.__doc__)
print(decimal_to_hexadecimal.__doc__)
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> & C:\Users\Admin\AppData\Local\Programs\Python\Python312\python.exe "c:/Users/Admin/Downloads/AI ASS
ISTENCE/conversion.py"
1010
0
10
0
FF
0

    This function takes a binary number as input and returns its decimal representation as an integer


    This function takes a decimal number as input and returns its binary representation as a string


    This function takes a decimal number as input and returns its hexadecimal representation as a string

PS C:\Users\Admin\Downloads\AI ASSISTENCE>
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc conversion
1010
0
10
0
FF
0

    This function takes a binary number as input and returns its decimal representation as an integer


    This function takes a decimal number as input and returns its binary representation as a string


    This function takes a decimal number as input and returns its hexadecimal representation as a string

Help on module conversion:

NAME
    conversion

FUNCTIONS
    binary_to_decimal(b)
        This function takes a binary number as input and returns its decimal representation as an integer

    decimal_to_binary(n)
        This function takes a decimal number as input and returns its binary representation as a string

    decimal_to_hexadecimal(n)
        This function takes a decimal number as input and returns its hexadecimal representation as a string

FILE
    c:\users\admin\downloads\ai assistence\conversion.py
```

```
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -w conversion
1010
0
10
0
FF
0

    This function takes a binary number as input and returns its decimal representation as an integer


    This function takes a decimal number as input and returns its binary representation as a string


    This function takes a decimal number as input and returns its hexadecimal representation as a string

wrote conversion.html
PS C:\Users\Admin\Downloads\AI ASSISTENCE> python -m pydoc -p 2429
Server ready at http://localhost:2429/
Server commands: [b]rowser, [q]uit
server> b
server>
```

Python 3.12.5 [tags/v3.12.5:ff3bc82, MSC v.1940 64 bit (AMD64)]
Windows-11

Module Index : Topics : Keywords
Get Search

Index of Modules

Built-in Modules

_abc	_imp	_statistics	
_ast	_io	_string	builtins
_bisect	_json	_struct	cmath
_blake2	_locale	_symtable	errno
_codecs	_lsprof	_thread	faulthandler
_codecs_cn	_md5	_tokenize	gc
_codecs_hk	_multibytecodec	_tracemalloc	itertools
_codecs_iso2022	_opcode	_typing	marshal
_codecs_jp	_operator	_warnings	math
_codecs_kr	_pickle	_weakref	mmap
_codecs_tw	_random	_winapi	msvcrt
_collections	_sha1	_xxinterpchannels	nt
_contextvars	_sha2	_xxsubinterpreters	sys
_csv	_sha3	array	time
_datetime	_signal	atexit	winreg
_functools	_sre	audioop	xxsubtype
_heapq	_stat	binascii	zlib

C:\Users\Admin\Downloads\AI ASSISTENCE

20th	aiac	even numbers	lab_8
24 feb	aiac_feb 17	feb_18	lab_ass_8
Untitled-1	attendance	jan 13th	labexam_2
Untitled-2	conversion	lab-7	labexamaiac

C:\Users\Admin\AppData\Local\Programs\Python\Python312\python312.zip

Python 3.12.5 [tags/v3.12.5:ff3bc82, MSC v.1940 64 bit (AMD64)]
Windows-11

dataclasses

Modules

_thread	functools	keyword
abc	inspect	re
copy	itertools	sys

Classes

builtins.AttributeError(builtins.Exception)
    FrozenInstanceError
builtins.object
    Field
    InitVar

class **Field**(builtins.object)

```
Field(default, default_factory, init, repr, hash, compare, metadata, kw_only)

# Instances of Field are only ever created from within this module,
# and only from the field() function, although Field instances are
# exposed externally as (conceptually) read-only objects.
#
# name and type are filled in after the fact, not in __init__.
# They're not known at the time this class is instantiated, but it's
# convenient if they're available later.
#
# When cls._FIELDS is filled in with a list of Field objects, the name
# and type fields will have been populated.
```

Methods defined here:

__init__(self, default, default_factory, init, repr, hash, compare, metadata, kw_only)
    Initialize self. See help(type(self)) for accurate signature.

Problem 5 – Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

Task5.py > ...

```python
"""
This module include three functions add_course(course_id,name,credits),remove_course(course_id),get_course(course_id) and their explanation
and also includes a docstring to explain the usage of each function
"""
courses = {}
def add_course(course_id, name, credits):
    """
    This function adds a course to the courses dictionary.

    Parameters:
    course_id (str): The unique identifier for the course.
    name (str): The name of the course.
    credits (int): The number of credits for the course.

    Returns:
    None
    """
    if course_id in courses:
        raise ValueError("Course ID already exists.")
    courses[course_id] = {"name": name, "credits": credits}
def remove_course(course_id):
    """
    This function removes a course from the courses dictionary.
    Parameters:
    course_id (str): The unique identifier for the course to be removed.
    Returns:
    None
    Raises:
    KeyError: If the course ID does not exist in the courses dictionary.
    """
    if course_id not in courses:
        raise KeyError("Course ID does not exist.")
    del courses[course_id]
def get_course(course_id):
    """
    This function retrieves the details of a course from the courses dictionary.
    Parameters:
    course_id (str): The unique identifier for the course to be retrieved.
    Returns:
    dict: A dictionary containing the name and credits of the course.
    Raises:
    KeyError: If the course ID does not exist in the courses dictionary.
    """
    if course_id not in courses:
        raise KeyError("Course ID does not exist.")
    return courses[course_id]
print(add_course.__doc__)
print(remove_course.__doc__)
print(get_course.__doc__)
import Task5
help(Task5)
```

```
PS D:\courses\AIAC\Lab_ass_9.1> python -m pydoc Task5

This function adds a course to the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:
None


This function removes a course from the courses dictionary.
Parameters:
course_id (str): The unique identifier for the course to be removed.
Returns:
None
Raises:
KeyError: If the course ID does not exist in the courses dictionary.


This function retrieves the details of a course from the courses dictionary.
Parameters:
course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the name and credits of the course.
Raises:
KeyError: If the course ID does not exist in the courses dictionary.
Help on module Task5:

NAME
    Task5

DESCRIPTION
    This module include three functions add_course(course_id,name,credits),remove_course(course_id),get_course(course_id) and their explanation    and also includes a docstring to explain the usage of each function

FUNCTIONS
    add_course(course_id, name, credits)
        This function adds a course to the courses dictionary.

        Parameters:
        course_id (str): The unique identifier for the course.
        name (str): The name of the course.
        credits (int): The number of credits for the course.

        Returns:
        None

    get_course(course_id)
        This function retrieves the details of a course from the courses dictionary.
        Parameters:
        course_id (str): The unique identifier for the course to be retrieved.
        Returns:
        dict: A dictionary containing the name and credits of the course.
        Raises:
        KeyError: If the course ID does not exist in the courses dictionary.

    remove_course(course_id)
        This function removes a course from the courses dictionary.
        Parameters:
        course_id (str): The unique identifier for the course to be removed.
-- More  --
```

127.0.0.1:5500/Task5.html

index

Task5 d:\courses\aiac\lab_ass_9.1\task5.py

This module include three functions add_course(course_id,name,credits),remove_course(course_id),get_course(course_id) and their explanation and also includes a docstring to explain the usage of each function

Modules

Task5

Functions

add_course(course_id, name, credits)

This function adds a course to the courses dictionary.

Parameters:
course_id (str): The unique identifier for the course.
name (str): The name of the course.
credits (int): The number of credits for the course.

Returns:
None

get_course(course_id)

This function retrieves the details of a course from the courses dictionary.
Parameters:
course_id (str): The unique identifier for the course to be retrieved.
Returns:
dict: A dictionary containing the name and credits of the course.
Raises:
KeyError: If the course ID does not exist in the courses dictionary.

remove_course(course_id)

This function removes a course from the courses dictionary.
Parameters:
course_id (str): The unique identifier for the course to be removed.
Returns:
None
Raises:
KeyError: If the course ID does not exist in the courses dictionary.

Data

courses = {}